

we manually deployed AWS instance scheduler in CloudFormation by clicking "deploy." from below AWS documentation below.

<https://docs.aws.amazon.com/solutions/instance-scheduler-on-aws/>

After testing the CF stack we deleted the stack and now we want to deploy this instance scheduler CF stack through terraform from management account of AWS organization so that the scheduler can run on all accounts listed, so how do we do through terraform like how we get yaml files etc and what are detailed steps to deploy using terraform

I'll outline a Terraform-based deployment that still uses AWS's official CloudFormation template, because Instance Scheduler on AWS is packaged as a CloudFormation solution. I'll also cover the management-account / member-account role setup so it can operate across AWS Organizations accounts.

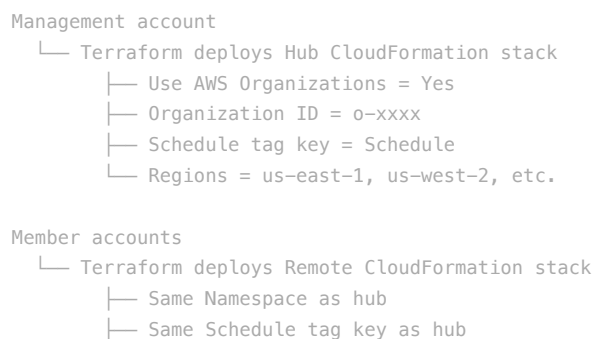
Thought for 34s >

You **do not need to convert the AWS Instance Scheduler YAML into native Terraform resources**. The cleanest approach is to let Terraform deploy the official AWS CloudFormation templates using `aws_cloudformation_stack` for the hub stack and either `aws_cloudformation_stack_set` or per-account assumed-role providers for the remote stacks.

AWS confirms this solution is delivered as CloudFormation templates: the main `instance-scheduler-on-aws.template` deploys the Lambda, DynamoDB table, EventBridge/CloudWatch rule, CloudWatch metrics, etc., and the `instance-scheduler-on-aws-remote.template` creates the cross-account role used by the hub to schedule spoke accounts. AWS Documentat...

Recommended design for your organization

Use this model:



- └─ Hub Account ID = management account ID
- └─ Same Region as hub stack

The remote stack **must be deployed in the same Region as the hub stack**, and AWS says to install the remote template only after the primary/hub stack is successfully installed. [AWS Documentat...](#)

Step 1: Get the official templates

You have two options.

Option A — Use AWS template URLs directly in Terraform

Use these inside Terraform code:

```
s.com/solutions-reference/instance-scheduler-on-aws/latest/instance-scheduler-on-aws.t  
s.com/solutions-reference/instance-scheduler-on-aws/latest/instance-scheduler-on-aws-re
```

Option B — Download the templates and store them in your repo

This is better for production because you can version-control exactly what you deployed.

```
mkdir -p templates

curl -o templates/instance-scheduler-on-aws.template \
  https://s3.amazonaws.com/solutions-reference/instance-scheduler-on-aws/latest/inst

curl -o templates/instance-scheduler-on-aws-remote.template \
  https://s3.amazonaws.com/solutions-reference/instance-scheduler-on-aws/latest/inst
```

Then in Terraform you can use `template_body = file(...)` instead of `template_url`.

Step 2: Check exact CloudFormation parameter names

AWS documentation shows the parameter labels, but Terraform needs the actual `ParameterKey` names. Before writing final Terraform, run this:

```
e-summary \
amazonaws.com/solutions-reference/instance-scheduler-on-aws/latest/instance-scheduler-on-

ParameterKey'
```

For the remote stack:

```
ry \
.com/solutions-reference/instance-scheduler-on-aws/latest/instance-scheduler-on-aws-rer
.
```

This is important because AWS can change parameter names between solution versions.

Step 3: Example Terraform folder structure

```
instance-scheduler-terraform/
  provider.tf
  variables.tf
  main.tf
  terraform.tfvars
  outputs.tf
```

Step 4: Terraform provider

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}

provider "aws" {
  region  = var.hub_region
  profile = var.management_profile
}
```

Example variables.tf :

```
variable "management_profile" {
  type        = string
  description = "AWS CLI profile for the management account"
}

variable "hub_region" {
  type        = string
  description = "Region where the Instance Scheduler hub stack will run"
  default     = "us-east-1"
```

```

}

variable "organization_id" {
  type        = string
  description = "AWS Organizations ID, for example o-xxxxxxxxxx"
}

variable "hub_account_id" {
  type        = string
  description = "Management or hub account ID"
}

variable "namespace" {
  type        = string
  default     = "org-scheduler"
}

variable "schedule_tag_key" {
  type        = string
  default     = "Schedule"
}

variable "scheduled_regions" {
  type        = list(string)
  default     = ["us-east-1"]
}

variable "remote_account_ids" {
  type        = list(string)
  description = "AWS member account IDs where remote stack must be deployed"
}

```

Example terraform.tfvars :

```

management_profile = "mgmt-account"
hub_region         = "us-east-1"
hub_account_id     = "111111111111"
organization_id    = "o-abc123xyz"

namespace          = "org-scheduler"
schedule_tag_key   = "Schedule"

scheduled_regions = [
  "us-east-1",
  "us-west-2"
]

remote_account_ids = [
  "222222222222",
  "333333333333",
  "444444444444"
]

```

Step 5: Deploy the hub stack from Terraform

```

locals {
  hub_template_url    = "https://s3.amazonaws.com/solutions-reference/instance-sched
  remote_template_url = "https://s3.amazonaws.com/solutions-reference/instance-sched

  scheduled_regions_csv = join(",", var.scheduled_regions)
}

resource "aws_cloudformation_stack" "instance_scheduler_hub" {
  name          = "instance-scheduler-hub"
  template_url = local.hub_template_url

  capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
  ]

  parameters = {
    Namespace                = var.namespace
    UsingAWSOrganizations    = "Yes"
    OrganizationId           = var.organization_id
    ScheduledServices         = "EC2,RDS"
    SchedulingInterval        = "5"
    DefaultTimezone           = "America/New_York"
    Regions                  = local.scheduled_regions_csv
    TagName                   = var.schedule_tag_key
    EnableCloudWatchMetrics   = "Yes"
    EnableCloudWatchLogs      = "No"
  }

  tags = {
    Application = "InstanceScheduler"
    ManagedBy   = "Terraform"
  }
}

```



The parameter names above are common examples, but you should adjust them based on the `get-template-summary` output from your exact template version. The hub stack has parameters such as Namespace, Use AWS Organizations, Organization ID/Remote Account IDs, Schedule tag key, default timezone, scheduling interval, and Regions.

[AWS Documentat...](#)

Also note: the template creates IAM resources, so Terraform/CloudFormation must acknowledge IAM capabilities. AWS CloudFormation requires `CAPABILITY_IAM` or `CAPABILITY_NAMED_IAM` when templates create IAM resources. [AWS Documentat...](#)

Step 6: Deploy remote stacks to member accounts

You have two practical options.

Option 1 — Use CloudFormation StackSets from Terraform

This is best when deploying to many accounts or OUs.

```

resource "aws_cloudformation_stack_set" "instance_scheduler_remote" {
  name          = "instance-scheduler-remote"

```

```

permission_model = "SERVICE_MANAGED"

template_url = local.remote_template_url

capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
]

parameters = {
    Namespace            = var.namespace
    UsingAWSOrganizations = "Yes"
    InstanceSchedulerAccount = var.hub_account_id
    Regions              = local.scheduled_regions_csv
    TagName               = var.schedule_tag_key
}

auto_deployment {
    enabled                = true
    retain_stacks_on_account_removal = false
}

depends_on = [
    aws_cloudformation_stack.instance_scheduler_hub
]
}

```

Then create StackSet instances for the member accounts:

```

resource "aws_cloudformation_stack_set_instance" "instance_scheduler_remote_accounts"
  for_each = toset(var.remote_account_ids)

  stack_set_name = aws_cloudformation_stack_set.instance_scheduler_remote.name
  account_id     = each.value
  region         = var.hub_region

  depends_on = [
    aws_cloudformation_stack_set.instance_scheduler_remote
  ]
}

```

CloudFormation StackSets with service-managed permissions are intended for deploying stacks to accounts managed by AWS Organizations, and AWS states that CloudFormation can create the required IAM roles on your behalf with this model.

AWS Documentat...

Option 2 — Assume role into each account and deploy normal stacks

Use this if your organization already has a standard cross-account deployment role such as `OrganizationAccountAccessRole`, `AWSControlTowerExecution`, or a custom Terraform execution role.

Example provider aliases:

```

provider "aws" {
  alias   = "member_222222222222"
  region = var.hub_region

  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/OrganizationAccountAccessRole"
  }
}

```

Then deploy the remote stack:

```

resource "aws_cloudformation_stack" "remote_222222222222" {
  provider      = aws.member_222222222222
  name          = "instance-scheduler-remote"
  template_url  = local.remote_template_url

  capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
  ]

  parameters = {
    Namespace           = var.namespace
    UsingAWSOrganizations = "Yes"
    InstanceSchedulerAccount = var.hub_account_id
    Regions              = local.scheduled_regions_csv
    TagName               = var.schedule_tag_key
  }

  depends_on = [
    aws_cloudformation_stack.instance_scheduler_hub
  ]
}

```

The remote stack creates the secondary-account permissions that allow the hub stack to schedule instances in those accounts. When AWS Organizations is enabled on both hub and remote stacks, AWS says no additional hub-stack update is required after deploying the remote stack. [AWS Documentat...](#)

Step 7: Apply Terraform

```

terraform init
terraform validate
terraform plan
terraform apply

```

After apply, check:

```

aws cloudformation describe-stacks \
  --stack-name instance-scheduler-hub \
  --region us-east-1

```

For StackSets:

```
aws cloudformation list-stack-instances \
  --stack-set-name instance-scheduler-remote \
  --region us-east-1
```

Step 8: Create schedules

After the hub stack is deployed, you still need schedules. You can create them in one of three ways:

1. Scheduler CLI
2. Direct DynamoDB configuration
3. CloudFormation custom resources

AWS's sample schedule shows that a schedule name becomes the tag value. For example, if the schedule is `london-working-hours`, the instance tag would be `Schedule=london-working-hours`. [AWS Documentat...](#)

Example using Scheduler CLI:

```
scheduler-cli create-period \
  --stack instance-scheduler-hub \
  --name weekdays-8-6 \
  --weekdays mon-fri \
  --begin-time 08:00 \
  --end-time 18:00

scheduler-cli create-schedule \
  --stack instance-scheduler-hub \
  --name business-hours-new-york \
  --periods weekdays-8-6 \
  --time-zone America/New_York
```

Then tag EC2/RDS resources:

```
Key:   Schedule
Value: business-hours-new-york
```

Step 9: Important cleanup because you deleted the test stack

Before redeploying, check if the previous stack retained resources. AWS's hub parameter **Retain data and logs** is enabled by default and can retain DynamoDB tables when the stack is deleted. [AWS Documentat...](#)

Check for leftover resources:

```
aws dynamodb list-tables --region us-east-1 | grep -i scheduler
aws logs describe-log-groups --region us-east-1 --log-group-name-prefix /aws/lambda
```



```
aws iam list-roles | grep -i scheduler
```

If you get naming conflicts during Terraform apply, either delete old retained resources or use a new namespace .

What I would recommend for your case

Since you want to deploy from the **management account** and run scheduler across listed AWS Organization accounts:

```
Use Terraform + CloudFormation Hub Stack in management account
Use Terraform + CloudFormation StackSet for remote stack in member accounts
Use AWS Organizations = Yes
Use same namespace/tag key everywhere
Use same Region for hub and remote stacks
Use schedules + resource tags to control what stops/starts
```

In simple words: **Terraform will not replace the AWS Instance Scheduler YAML. Terraform will deploy and manage that CloudFormation YAML as infrastructure code.**

      ...  Sources

still not able to understand, can you explain clearly how generic yaml workflow will be when we use Terraform for deployment and what can be solution for Instance Scheduler

Think of Terraform here as the “deployment controller,” not as a replacement for the AWS YAML. The YAML still comes from AWS; Terraform’s job is to pass that YAML to CloudFormation with the right parameters and track it in Terraform state.

Thought for 8s >

Yes — let’s make it very simple.

You have **two IaC layers** here:

```
Layer 1: Terraform
└─ Calls CloudFormation and passes parameters

Layer 2: AWS CloudFormation YAML
└─ Actually creates the Instance Scheduler resources
```

So Terraform is **not replacing the YAML**. Terraform is only saying:

“AWS CloudFormation, please deploy this AWS-provided YAML template with these parameter values.”

AWS Instance Scheduler is officially provided as CloudFormation templates. AWS has a **hub template** and a **remote template**. The hub stack creates the scheduler Lambda, DynamoDB configuration table, EventBridge rule, IAM roles, etc. The remote stack creates permissions in each secondary/member account so the hub can stop/start EC2 and RDS there. [AWS Documentat...](#) **+1**

1. Generic YAML workflow with Terraform

Normally, when you manually click **Deploy** from AWS documentation, this happens:

```
AWS documentation
↓
Launch CloudFormation template
↓
You enter parameters manually
↓
CloudFormation creates resources
```

When using Terraform, the workflow becomes:

```
Terraform code
↓
References AWS CloudFormation YAML URL
↓
Passes parameters automatically
↓
CloudFormation creates resources
↓
Terraform tracks the CloudFormation stack
```

So instead of clicking this manually:

```
Deploy → Next → Enter Parameters → Create Stack
```

Terraform does this:

```
hcl

resource "aws_cloudformation_stack" "example" {
  name          = "my-stack"
  template_url = "https://aws-template-url/template.yaml"

  parameters = {
    Parameter1 = "value1"
    Parameter2 = "value2"
  }
}
```

That is the main idea.

2. For Instance Scheduler, there are two YAML templates

For AWS Instance Scheduler, think like this:

```
Hub template
└─ Deploy once in management / hub account
```

```
Remote template
└─ Deploy in every member account that should be scheduled
```

AWS says the remote stack must be deployed in the **same Region as the hub stack**.
AWS also says to deploy the remote template only after the hub stack is successfully installed. [AWS Documentat...](#)

Example:

```
Management account: 111111111111
Region: us-east-1

Deploy:
    instance-scheduler-on-aws.template
```

Then in member accounts:

```
Account: 222222222222
Account: 333333333333
Account: 444444444444

Deploy:
    instance-scheduler-on-aws-remote.template
```

3. Recommended solution for your case

Because you want to deploy from the **management account** for multiple AWS Organization accounts, the best solution is:

```
Terraform in management account
└─ Deploy Hub CloudFormation stack
└─ Deploy Remote CloudFormation stack using StackSets to member accounts
```

Architecture:

```
AWS Organization
├─ Management Account
│   └─ Instance Scheduler Hub Stack
│       ├── Lambda scheduler
│       ├── DynamoDB schedule table
│       ├── EventBridge rule
│       └─ IAM role to assume into member accounts
├─ Member Account 1
│   └─ Instance Scheduler Remote Stack
│       └─ IAM role trusted by Hub Account
├─ Member Account 2
│   └─ Instance Scheduler Remote Stack
│       └─ IAM role trusted by Hub Account
└─ Member Account 3
```

- └─ Instance Scheduler Remote Stack
 - └─ IAM role trusted by Hub Account

The hub stack looks for resources tagged like:

Key: Schedule
Value: office-hours

Then it starts/stops EC2 or RDS based on the schedule defined in the scheduler configuration.

4. Simple Terraform example

main.tf

```
hcl

provider "aws" {
  region = "us-east-1"
}

locals {
  hub_template_url    = "https://s3.amazonaws.com/solutions-reference/instance-sched
  remote_template_url = "https://s3.amazonaws.com/solutions-reference/instance-sched
}

resource "aws_cloudformation_stack" "instance_scheduler_hub" {
  name          = "instance-scheduler-hub"
  template_url  = local.hub_template_url

  capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
  ]

  parameters = {
    Namespace           = "org-scheduler"
    UsingAWSOrganizations = "Yes"
    OrganizationId      = "o-xxxxxxxxxx"
    ScheduledServices    = "EC2,RDS"
    SchedulingInterval   = "5"
    DefaultTimezone      = "America/New_York"
    Regions              = "us-east-1"
    TagName              = "Schedule"
  }
}
```

This is equivalent to clicking **Deploy** for the hub template manually.

5. Remote account deployment using StackSets

For the member accounts, use CloudFormation StackSets.

hcl

```

resource "aws_cloudformation_stack_set" "instance_scheduler_remote" {
  name          = "instance-scheduler-remote"
  permission_model = "SERVICE_MANAGED"

  template_url = local.remote_template_url

  capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
  ]

  parameters = {
    Namespace          = "org-scheduler"
    UsingAWSOrganizations = "Yes"
    InstanceSchedulerAccount = "111111111111"
  }

  auto_deployment {
    enabled          = true
    retain_stacks_on_account_removal = false
  }

  depends_on = [
    aws_cloudformation_stack.instance_scheduler_hub
  ]
}

```

Then deploy the remote stack to selected member accounts:

```

hcl

resource "aws_cloudformation_stack_set_instance" "member_accounts" {
  for_each = toset([
    "222222222222",
    "333333333333",
    "444444444444"
  ])

  stack_set_name = aws_cloudformation_stack_set.instance_scheduler_remote.name
  account_id     = each.value
  region        = "us-east-1"

  depends_on = [
    aws_cloudformation_stack_set.instance_scheduler_remote
  ]
}

```

This is equivalent to logging into each member account and clicking **Deploy remote stack**, but Terraform and StackSets do it centrally.

6. The end-to-end workflow

Your deployment workflow should be:

- Step 1
Create Terraform repo.
- Step 2
Terraform references AWS official hub YAML template.

Step 3
Terraform deploys hub stack in management account.

Step 4
Terraform references AWS official remote YAML template.

Step 5
Terraform deploys remote stack to member accounts using StackSets.

Step 6
Create scheduler periods/schedules.

Step 7
Tag EC2/RDS resources in member accounts.

Step 8
Scheduler starts/stops tagged resources.

7. How schedules actually work

After deployment, the scheduler does nothing until schedules exist and resources are tagged.

Example:

```
Schedule name:
office-hours

Period:
Monday-Friday
Start: 08:00
Stop: 18:00
Timezone: America/New_York
```

Then tag EC2/RDS:

```
Tag key:   Schedule
Tag value: office-hours
```

AWS provides sample schedules where the schedule name becomes the tag value on the instance. [AWS Documentat...](#)

So the process is:

```
Create schedule called office-hours
↓
Tag EC2 or RDS with Schedule=office-hours
↓
Instance Scheduler reads the tag
↓
Scheduler starts/stops resource based on that schedule
```

8. Important point about YAML files

You asked: "How do we get YAML files?"

You have 2 choices.

Option A — Reference AWS URL directly

This is easiest:

```
hcl

template_url = "https://s3.amazonaws.com/solutions-reference/instance-scheduler-on-a
```

Terraform tells CloudFormation:

"Use this AWS-hosted template."

Option B — Download YAML into your Git repo

This is better for change control:

```
repo/
  templates/
    instance-scheduler-on-aws.template
    instance-scheduler-on-aws-remote.template
  main.tf
```

Then Terraform uses:

```
hcl

template_body = file("${path.module}/templates/instance-scheduler-on-aws.template")
```

I recommend **Option B for production** because your team can review and version the exact template being deployed.

9. Best solution statement you can use internally

You can explain it like this:

Edit

We will deploy AWS Instance Scheduler using Terraform as the orchestration layer while continuing to use the AWS-provided CloudFormation templates. Terraform will create and manage the CloudFormation hub stack in the management account and deploy the remote stack to member accounts using CloudFormation StackSets.

The hub stack will contain the scheduler Lambda, DynamoDB schedule configuration, EventBridge rule, and central IAM permissions. The remote stack will be deployed in each member account to create the cross-account role

required for the hub scheduler to start and stop EC2/RDS resources. Resources will be controlled through tags such as Schedule=office-hours.

10. Final recommendation

For your use case, do this:

```
Use Terraform
+
AWS official Instance Scheduler CloudFormation templates
+
CloudFormation StackSets for member accounts
+
AWS Organizations integration
```

Do **not** try to convert the Instance Scheduler YAML into native Terraform. That will be difficult to maintain and unnecessary. The right solution is:

```
Terraform manages CloudFormation.
CloudFormation deploys Instance Scheduler.
Instance Scheduler manages EC2/RDS start-stop based on tags.
```

      ...  Sources

how terraform is generally used in many orgs

Thought for a couple of seconds >

In many organizations, **Terraform is used as the standard way to create, update, and manage cloud infrastructure through code** instead of manually clicking in the AWS Console.

Think of Terraform as:

```
A repeatable deployment tool for infrastructure
```

Instead of this:

```
Login to AWS Console
Click EC2
Create VPC
Create subnet
Create IAM role
Create Lambda
Create RDS
Manually repeat in dev/test/prod
```

Organizations do this:


```
Write Terraform code
Review code in Git
Run terraform plan
Approve changes
Run terraform apply
Terraform creates/updates infrastructure
```

1. Common Terraform workflow in organizations

A typical workflow looks like this:

```
Developer / Cloud Engineer writes Terraform code
↓
Code is committed to Git repo
↓
Pull request is opened
↓
Team reviews the infrastructure change
↓
CI/CD pipeline runs terraform plan
↓
Approver reviews the plan
↓
Pipeline runs terraform apply
↓
AWS resources are created or updated
```

So Terraform becomes part of the same process as application code.

2. Example

Instead of manually creating an S3 bucket, an engineer writes:

```
hcl

resource "aws_s3_bucket" "logs" {
  bucket = "company-prod-logs-12345"
}
```

Then runs:

```
Bash

terraform plan
terraform apply
```

Terraform creates the bucket in AWS.

Later, if someone changes the bucket policy, lifecycle rule, encryption, or tags, Terraform tracks that change.

3. Why organizations use Terraform

Main reasons:

Consistency

Same infrastructure can be deployed to dev, test, prod.

Repeatability

No need to remember manual console steps.

Auditability

Changes are reviewed in Git pull requests.

Governance

Security, tagging, naming, and cost rules can be standardized.

Automation

Infrastructure can be deployed through CI/CD pipelines.

Disaster recovery

If something is deleted, infrastructure can be recreated from code.

4. How Terraform is organized in companies

Most organizations have separate Terraform folders or repositories by environment.

Example:

```
terraform/  
  modules/  
    vpc/  
    ec2/  
    rds/  
    lambda/  
    iam/  
    cloudformation-stack/  
  environments/  
    dev/  
    test/  
    prod/
```

The **module** is reusable code.

The **environment** calls that module with different values.

[↑ Share](#) ...

```
VPC module  
├─ Used by dev  
├─ Used by test  
└─ Used by prod
```

So dev may use small subnets, while prod may use larger subnets.

5. What are Terraform modules?

A module is a reusable Terraform package.

Example:

```
module "vpc" {  
  source = "../../modules/vpc"  
  
  environment = "prod"  
  cidr_block  = "10.10.0.0/16"  
}
```

The company creates one standard VPC module and all teams use it. This prevents every team from creating VPCs differently.

Common modules:

```
VPC module  
EC2 module  
RDS module  
S3 module  
IAM role module  
Lambda module  
CloudWatch alarm module  
CloudFormation stack module
```

For your Instance Scheduler case, your organization could create a module like:

```
modules/  
  instance-scheduler/
```

Then call it from the management account Terraform code.

6. How Terraform state is used

Terraform keeps a record of what it created. This is called the **state file**.

For example, if Terraform created:

```
VPC  
Subnets  
IAM roles  
Lambda  
CloudFormation stack
```

Terraform state remembers those resources.

In organizations, the state file is usually stored remotely, not on someone's laptop.

Common AWS setup:

```
Terraform state file → S3 bucket  
State locking       → DynamoDB table
```

Example backend:

```
hcl
```

```

terraform {
  backend "s3" {
    bucket      = "company-terraform-state"
    key         = "instance-scheduler/prod/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}

```

This helps multiple engineers work safely without overwriting each other.

7. How Terraform is used with AWS Organizations

In large AWS environments, Terraform is often run from a central account, such as:

```

Management account
Shared services account
Deployment account
Platform account

```

From there, Terraform assumes roles into other AWS accounts.

Example:

```

Terraform runs in management account
      ↓
Assumes role into dev account
      ↓
Deploys resources

Terraform runs in management account
      ↓
Assumes role into test account
      ↓
Deploys resources

Terraform runs in management account
      ↓
Assumes role into prod account
      ↓
Deploys resources

```

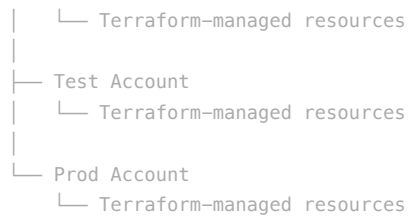
This avoids logging into every account manually.

8. Generic AWS Organizations Terraform model

```

AWS Organization
|
├─ Management Account
|   └─ Terraform pipeline / admin deployment
|
├─ Shared Services Account
|   └─ Central tools, logging, monitoring
|
└─ Dev Account

```



Terraform can deploy into these accounts by using IAM role assumption:

```
hcl

provider "aws" {
  alias = "prod"
  region = "us-east-1"

  assume_role {
    role_arn = "arn:aws:iam::123456789012:role/TerraformExecutionRole"
  }
}
```

9. How CI/CD is usually used

Many organizations do not run Terraform manually from laptops for production.

They use tools like:

```
GitHub Actions
GitLab CI
Jenkins
Azure DevOps
AWS CodePipeline
Terraform Cloud
Atlantis
Spacelift
env0
```

Typical pipeline:

```
Pull request opened
↓
terraform fmt
↓
terraform validate
↓
terraform plan
↓
Team reviews plan
↓
Approval
↓
terraform apply
```

This gives control and audit history.

10. Terraform for CloudFormation-based AWS solutions

This is exactly your Instance Scheduler situation.

Some AWS solutions are not built as normal Terraform modules. They are provided as CloudFormation templates.

Examples:

```
AWS Instance Scheduler
AWS Security Hub automation solutions
AWS Landing Zone older patterns
AWS Solutions Library templates
```

In that case, organizations often use Terraform like this:

```
hcl

resource "aws_cloudformation_stack" "solution" {
  name       = "instance-scheduler"
  template_url = "https://aws-solution-template-url"

  parameters = {
    Namespace = "org-scheduler"
    TagName    = "Schedule"
  }

  capabilities = [
    "CAPABILITY_IAM",
    "CAPABILITY_NAMED_IAM"
  ]
}
```

So Terraform still manages the deployment, but CloudFormation creates the detailed AWS solution resources.

That means:

```
Terraform owns the CloudFormation stack
CloudFormation owns the resources inside the stack
```

This is common.

11. How this applies to Instance Scheduler

For your case, the organizational Terraform approach would be:

```
Terraform repo
↓
Deploy Instance Scheduler hub stack in management account
↓
Deploy remote stack to member accounts using StackSets
↓
Create schedules
↓
Tag EC2/RDS resources
↓
Scheduler starts/stops resources
```

The code may be organized like this:

```
terraform-instance-scheduler/  
  backend.tf  
  providers.tf  
  variables.tf  
  main.tf  
  outputs.tf  
  terraform.tfvars
```

And the deployment logic would be:

```
main.tf  
├── aws_cloudformation_stack  
│   └── Hub stack  
├── aws_cloudformation_stack_set  
│   └── Remote stack definition  
└── aws_cloudformation_stack_set_instance  
    └── Deploy remote stack to member accounts
```

12. Simple way to explain to your team

You can say:

Edit

Terraform is generally used in organizations to manage infrastructure as code. Instead of creating AWS resources manually through the console, teams define the required infrastructure in Terraform files, store the code in Git, review changes through pull requests, and deploy through a controlled pipeline.

For AWS Instance Scheduler, Terraform will not replace the AWS-provided CloudFormation template. Terraform will be used to deploy and manage the CloudFormation hub stack in the management account and the remote stack in member accounts, likely through CloudFormation StackSets. This gives us repeatable deployment, version control, auditability, and easier management across multiple AWS accounts.

13. In one sentence

Terraform is usually used as the **central, version-controlled, repeatable deployment method** for infrastructure, while AWS Console manual changes are avoided except for testing or emergency troubleshooting.

For your Instance Scheduler, the right model is:

```
Terraform = deployment and governance layer  
CloudFormation YAML = AWS solution template  
Instance Scheduler = actual automation that stops/starts EC2 and RDS
```

